

Problem Description:

You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet.

You are required to answer Q queries of type u, c , where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries.

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq u, v \leq N$$

- c is a lowercase alphabet
- s_i is a lowercase alphabet for all $1 \leq i \leq N$
- 1 is the root node

Input format

- First line: Two space-separated integers N and Q respectively
- Second line: A string s of length N (where the i^{th} character of s represents the character stored in node i)
- Next $N - 1$ line: Two space-separated integers u and v denoting an edge between node u and node v
- Next Q lines: An integer u and a space-separated character c

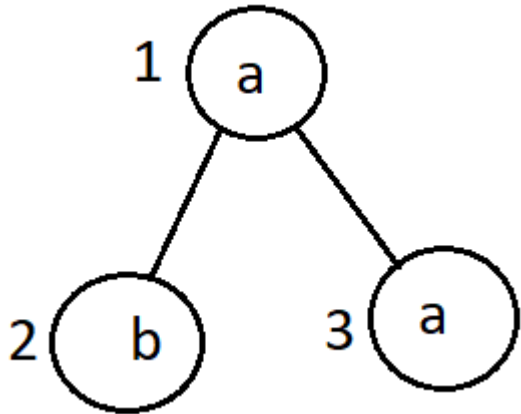
Output format

For each query, print the output in a new line.

Note: It is guaranteed that the input generates a valid tree.

Explanation for test case 1

Tree given in the sample input will look like that



Number of nodes in the subtree of node 1 having 'a' stored in it is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

3 1
aba
1 2
1 3
1 a

EXPECTED OUTPUT

2

Test Case 2

INPUT (STDIN)

4 1
wsx
1 2
1 3
1 1
1 w

EXPECTED OUTPUT

10

Mandatory Test Cases

Test Case 1

KEYWORD

while(0--)

Test Case 2

KEYWORD

for(i = 0; i < N-1; i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

49

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 typedef struct {
7     int *head;
8     int *to;
9     int *nxt;
10    int edge_cnt;
11} Graph;
12
13void init_graph(Graph *g, int n) {
14    g->head = (int*)malloc((n + 1) * sizeof(int));
15    for (int i = 0; i <= n; i++) g->head[i] = -1;
16    g->to = (int*)malloc(2 * (n + 5) * sizeof(int));
17    g->nxt = (int*)malloc(2 * (n + 5) * sizeof(int));
18    g->edge_cnt = 0;
19}
20
21void add_edge(Graph *g, int u, int v) {
22    g->to[g->edge_cnt] = v;
23    g->nxt[g->edge_cnt] = g->head[u];
24    g->head[u] = g->edge_cnt++;
25}
```

```

26
27int timer = 0;
28int *tin, *tout;
29int *char_list[26];
30int char_count[26];
31int char_cap[26];
32char *s;
33
34void dfs(int u, int p, Graph *g) {
35    tin[u] = ++timer;
36    int c = s[u - 1] - 'a';
37    if (c >= 0 && c < 26) {
38        if (char_count[c] >= char_cap[c]) {
39            char_cap[c] = char_cap[c] == 0 ? 16 : char_cap[c] * 2;
40            char_list[c] = (int*)realloc(char_list[c], char_cap[c] * sizeof(int));
41        }
42        char_list[c][char_count[c]++] = tin[u];
43    }
44    for (int e = g->head[u]; e != -1; e = g->nxt[e]) {
45        int v = g->to[e];
46        if (v != p) {

```

```

47         dfs(v, u, g);
48     }
49 }
50 tout[u] = timer;
51}
52
53int lower_bound(int *arr, int n, int val) {
54     int l = 0, r = n;
55     while (l < r) {
56         int mid = (l + r) / 2;
57         if (arr[mid] >= val) r = mid;
58         else l = mid + 1;
59     }
60     return l;
61}
62
63int upper_bound(int *arr, int n, int val) {
64     int l = 0, r = n;

```

main | CH.SC.U4CSE25225 | Tree29.c

Ln 128, Col 1 Spaces: 4 UTF-8 CRLF C

```

66         int mid = (l + r) / 2;
67         if (arr[mid] > val) r = mid;

```

```

68         else l = mid + 1;
69     }
70     return l;
71}
72
73int main() {
74     int N, Q;
75     if (scanf("%d %d", &N, &Q) != 2) return 0;
76
77     s = (char*)malloc((N + 5) * sizeof(char));
78     if (scanf("%s", s) != 1) return 0;
79
80     Graph g;
81     init_graph(&g, N);
82
83     int i;
84     for(i = 0; i < N-1; i++) {
85         int u, v;
86         if (scanf("%d %d", &u, &v) == 2) {
87             add_edge(&g, u, v);
88             add_edge(&g, v, u);

```

```

89     }
90 }
91
92 tin = (int*)malloc((N + 1) * sizeof(int));
93 tout = (int*)malloc((N + 1) * sizeof(int));
94 for (int c = 0; c < 26; c++) {
95     char_list[c] = NULL;
96     char_count[c] = 0;
97     char_cap[c] = 0;
98 }
99
100 dfs(1, 0, &g);
101
102 while(Q--) {
103     int u;
104     char ch[10];
105     if (scanf("%d %s", &u, ch) == 2) {
106         int c = ch[0] - 'a';
107         if (c < 0 || c >= 26) {
108             printf("0\n");

```

```

109         } else {
110             int l = lower_bound(char_list[c], char_count[c], tin[u]);
111             int r = upper_bound(char_list[c], char_count[c], tout[u]);
112             printf("%d\n", r - l);
113         }
114     }
115 }
116
117 free(s);
118 free(tin);
119 free(tout);
120 for (int c = 0; c < 26; c++) {
121     if (char_list[c]) free(char_list[c]);
122 }
123 free(g.head);
124 free(g.to);
125 free(g.nxt);
126 return 0;
127
128

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 1

aba

1 2

1 3

1 a

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree29.exe

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 1

wsx

1 2

1 3

1 1

1 w

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree29.exe

1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>